

Ordenação em tempo linear

Aula 7

Fábio Henrique Viduani Martinez

Faculdade de Computação
Universidade Federal de Mato Grosso do Sul

Projeto e Análise de Algoritmos I

Conteúdo da aula

- 1 Introdução
- 2 Limitante inferior para ordenação
- 3 Ordenação por contagem
- 4 Ordenação por colunas
- 5 Ordenação em “baldes”
- 6 Exercícios

Motivação

- ▶ vimos vários algoritmos que podem ordenar n números em tempo $O(n \lg n)$
- ▶ MERGESORT e HEAPSORT obtêm esse limitante superior no pior caso; QUICKSORT o obtém no caso médio
- ▶ além disso, para cada um desses algoritmos, podemos produzir uma sequência de entrada de n números que faz com que o algoritmo tenha tempo de execução $\Omega(n \lg n)$
- ▶ esses algoritmos compartilham uma propriedade interessante: determinam a sequência ordenada baseados apenas na comparação entre os elementos de entrada
- ▶ são chamados de **algoritmos de ordenação por comparação**

Motivação

- ▶ veremos que qualquer algoritmo de ordenação por comparação deve fazer $\Omega(n \lg n)$ comparações no pior caso para ordenar n elementos
- ▶ dessa forma, MERGESORT e HEAPSORT são assintoticamente ótimos e não existe um algoritmo de ordenação por comparação que seja mais rápido
- ▶ veremos também algoritmos de ordenação chamados COUNTINGSORT, RADIXSORT e BUCKETSORT que têm tempo de execução linear

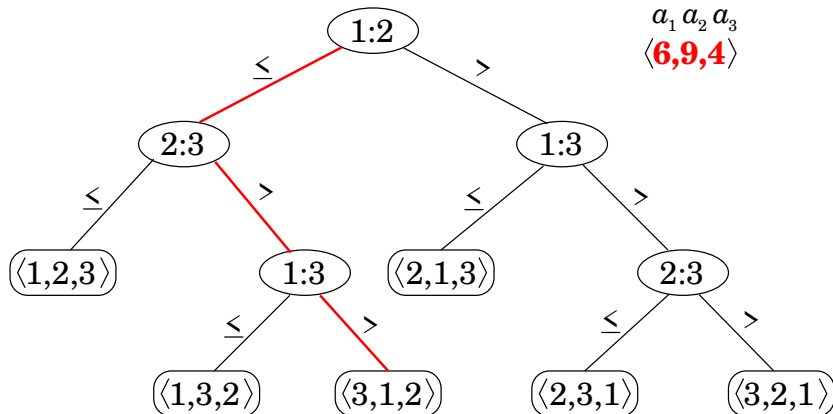
Preliminares

- ▶ em um algoritmo de ordenação por comparação, usamos apenas comparações entre elementos para obter uma ordem dos elementos de entrada $\langle a_1, a_2, \dots, a_n \rangle$
- ▶ isto é, dados dois elementos a_i e a_j , efetuamos um dos testes $a_i < a_j, a_i \leq a_j, a_i = a_j, a_i \geq a_j$ ou $a_i > a_j$ para determinar sua ordem relativa
- ▶ suponha que todos os elementos de entrada são distintos
- ▶ dessa forma, a comparação $a_i = a_j$ não é útil
- ▶ observe ainda que as comparações $a_i < a_j, a_i \leq a_j, a_i \geq a_j$ e $a_i > a_j$ são equivalentes, já que fornecem o mesmo tipo de informação sobre a ordem relativa de a_i e a_j
- ▶ portanto, consideramos que todas as comparações têm a forma $a_i \leq a_j$

Árvores de decisão

- ▶ algoritmos de ordenação por comparação podem ser vistos abstratamente em termos de árvores de decisão
- ▶ uma **árvore de decisão** é uma árvore binária cheia que representa as comparações entre os elementos que são executadas por um algoritmo de ordenação sobre uma entrada de um dado tamanho

Árvores de decisão



Limitante inferior para o pior caso

- ▶ o comprimento de um caminho mais longo da raiz de uma árvore de decisão a qualquer uma de suas folhas representa o número total de comparações, no pior caso, executadas por um algoritmo de ordenação
- ▶ portanto, o número de comparações, no pior caso, para um dado algoritmo de ordenação por comparação é igual à altura de sua árvore de decisão
- ▶ o limitante inferior sobre as alturas de todas as árvores de decisão nas quais cada permutação aparece como uma folha é, assim, um limitante inferior sobre o tempo de execução de qualquer algoritmo de ordenação por comparação

Limitante inferior para o pior caso

Teorema

Qualquer algoritmo de ordenação por comparação realiza $\Omega(n \lg n)$ comparações no pior caso.

Prova

Considere uma árvore de decisão de altura h com f folhas, associada a um algoritmo de ordenação por comparação sobre n elementos.

Como cada uma das $n!$ permutações de entrada ocorre em uma folha, então $n! \leq f$. Como uma árvore binária de altura h tem no máximo 2^h folhas, temos

$$n! \leq f \leq 2^h,$$

e assim

$$n! \leq 2^h \Rightarrow \lg(n!) \leq \lg 2^h \Rightarrow h \geq \lg(n!) \Rightarrow h = \Omega(n \lg n).$$

Limitante inferior para o pior caso

Teorema

Qualquer algoritmo de ordenação por comparação realiza $\Omega(n \lg n)$ comparações no pior caso.

Prova

Considere uma árvore de decisão de altura h com f folhas, associada a um algoritmo de ordenação por comparação sobre n elementos. Como cada uma das $n!$ permutações de entrada ocorre em uma folha, então $n! \leq f$. Como uma árvore binária de altura h tem no máximo 2^h folhas, temos

$$n! \leq f \leq 2^h,$$

e assim

$$n! \leq 2^h \Rightarrow \lg(n!) \leq \lg 2^h \Rightarrow h \geq \lg(n!) \Rightarrow h = \Omega(n \lg n).$$

Limitante inferior para o pior caso

Teorema

Qualquer algoritmo de ordenação por comparação realiza $\Omega(n \lg n)$ comparações no pior caso.

Prova

Considere uma árvore de decisão de altura h com f folhas, associada a um algoritmo de ordenação por comparação sobre n elementos. Como cada uma das $n!$ permutações de entrada ocorre em uma folha, então $n! \leq f$. Como uma árvore binária de altura h tem no máximo 2^h folhas, temos

$$n! \leq f \leq 2^h,$$

e assim

$$n! \leq 2^h \Rightarrow \lg(n!) \leq \lg 2^h \Rightarrow h \geq \lg(n!) \Rightarrow h = \Omega(n \lg n).$$

Limitante inferior para o pior caso

Teorema

Qualquer algoritmo de ordenação por comparação realiza $\Omega(n \lg n)$ comparações no pior caso.

Prova

Considere uma árvore de decisão de altura h com f folhas, associada a um algoritmo de ordenação por comparação sobre n elementos. Como cada uma das $n!$ permutações de entrada ocorre em uma folha, então $n! \leq f$. Como uma árvore binária de altura h tem no máximo 2^h folhas, temos

$$n! \leq f \leq 2^h,$$

e assim

$$n! \leq 2^h \Rightarrow \lg(n!) \leq \lg 2^h \Rightarrow h \geq \lg(n!) \Rightarrow h = \Omega(n \lg n).$$

Limitante inferior para o pior caso

Teorema

Qualquer algoritmo de ordenação por comparação realiza $\Omega(n \lg n)$ comparações no pior caso.

Prova

Considere uma árvore de decisão de altura h com f folhas, associada a um algoritmo de ordenação por comparação sobre n elementos. Como cada uma das $n!$ permutações de entrada ocorre em uma folha, então $n! \leq f$. Como uma árvore binária de altura h tem no máximo 2^h folhas, temos

$$n! \leq f \leq 2^h,$$

e assim

$$n! \leq 2^h \Rightarrow \lg(n!) \leq \lg 2^h \Rightarrow h \geq \lg(n!) \Rightarrow h = \Omega(n \lg n).$$

Limitante inferior para o pior caso

Corolário

HEAPSORT e MERGESORT são algoritmos de ordenação por comparação assintoticamente ótimos.

Prova

Os limitantes superiores $O(n \lg n)$ dos tempos de execução dos algoritmos HEAPSORT e MERGESORT casam com o limitante inferior de pior caso $\Omega(n \lg n)$ do teorema anterior.

Preliminares

- ▶ suponha que cada um dos n elementos da entrada é um número no intervalo de 0 a k , para algum número inteiro k
- ▶ quando $k = O(n)$ o algoritmo COUNTINGSORT tem tempo de execução $\Theta(n)$
- ▶ para cada elemento da entrada x , o algoritmo determina o número de elementos menores que x
- ▶ o algoritmo usa então essa informação para armazenar o elemento x diretamente na sua posição no vetor de saída

Algoritmo COUNTINGSORT

COUNTINGSORT(A, B, k)

01. seja $C[0..k]$ um novo vetor

02. **para** $i \leftarrow 0$ **até** k

03. $C[i] \leftarrow 0$

04. **para** $j \leftarrow 1$ **até** $A.length$

05. $C[A[j]] \leftarrow C[A[j]] + 1$

06. // agora, $C[i]$ contém o número de elementos iguais a i

07. **para** $i \leftarrow 1$ **até** k

08. $C[i] \leftarrow C[i] + C[i - 1]$

09. // agora, $C[i]$ contém o número de elementos menores ou iguais a i

10. **para** $j \leftarrow A.length$ **até** 1 **passo** -1

11. $B[C[A[j]]] \leftarrow A[j]$

12. $C[A[j]] \leftarrow C[A[j]] - 1$

Exemplo de execução do COUNTINGSORT

A

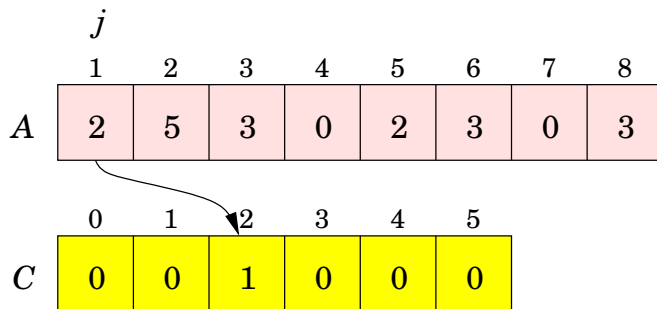
1	2	3	4	5	6	7	8
2	5	3	0	2	3	0	3

Exemplo de execução do COUNTINGSORT

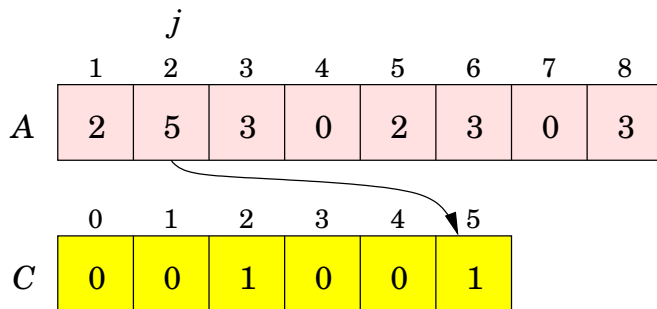
	1	2	3	4	5	6	7	8
<i>A</i>	2	5	3	0	2	3	0	3

	0	1	2	3	4	5
<i>C</i>	0	0	0	0	0	0

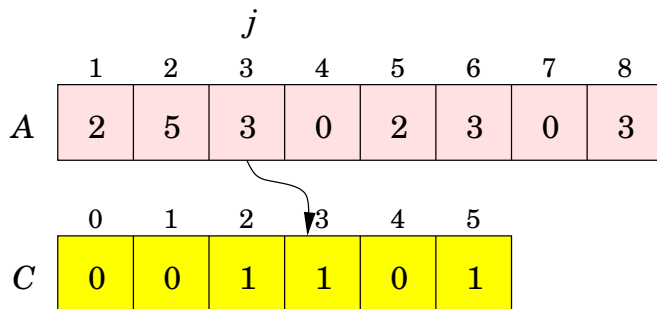
Exemplo de execução do COUNTINGSORT



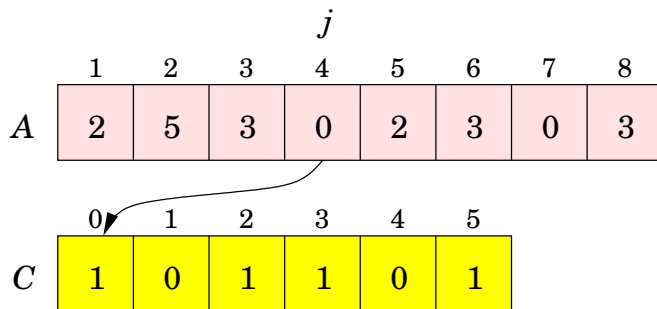
Exemplo de execução do COUNTINGSORT



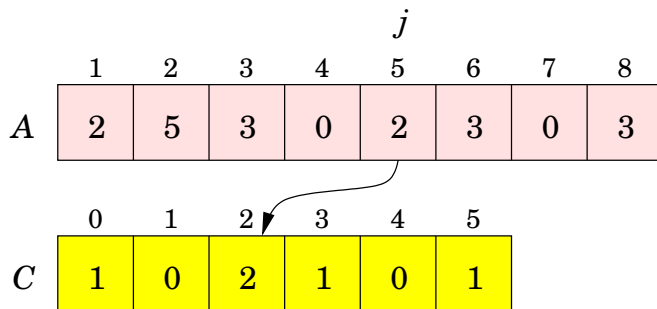
Exemplo de execução do COUNTINGSORT



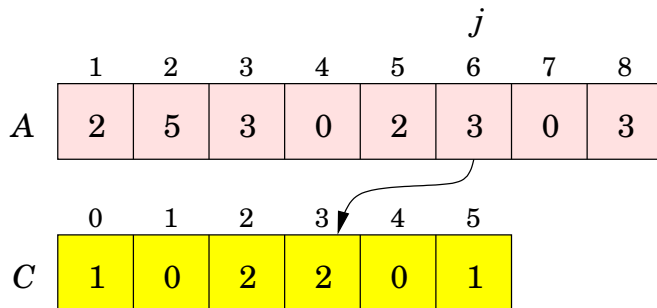
Exemplo de execução do COUNTINGSORT



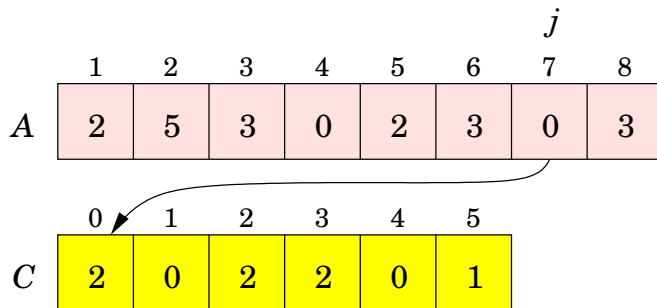
Exemplo de execução do COUNTINGSORT



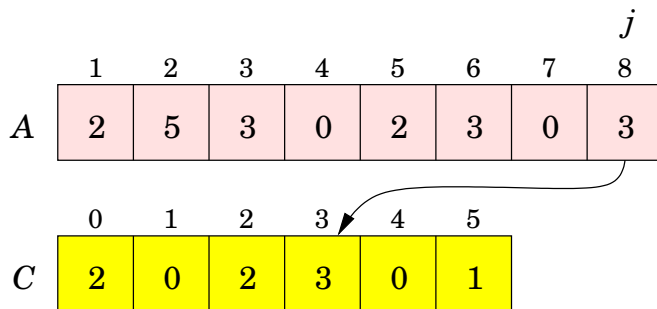
Exemplo de execução do COUNTINGSORT



Exemplo de execução do COUNTINGSORT



Exemplo de execução do COUNTINGSORT

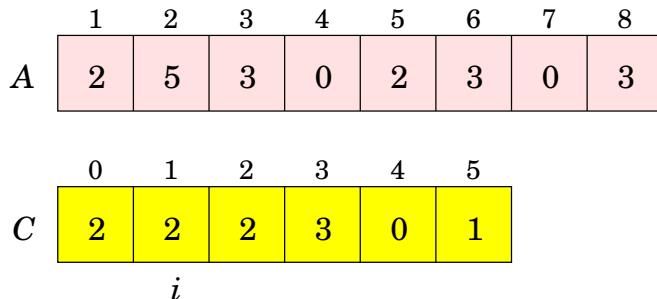


Exemplo de execução do COUNTINGSORT

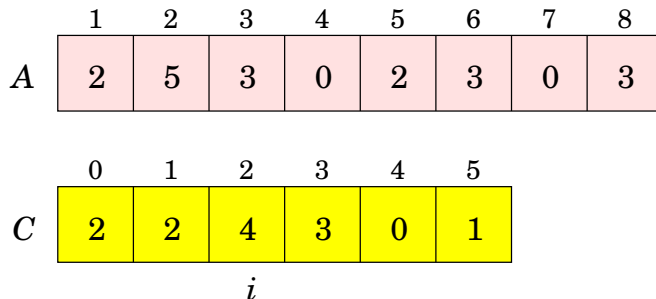
	j							
	1	2	3	4	5	6	7	8
A	2	5	3	0	2	3	0	3

	0	1	2	3	4	5
C	2	0	2	3	0	1

Exemplo de execução do COUNTINGSORT



Exemplo de execução do COUNTINGSORT



Exemplo de execução do COUNTINGSORT

	1	2	3	4	5	6	7	8
<i>A</i>	2	5	3	0	2	3	0	3

	0	1	2	3	4	5
<i>C</i>	2	2	4	7	0	1

i

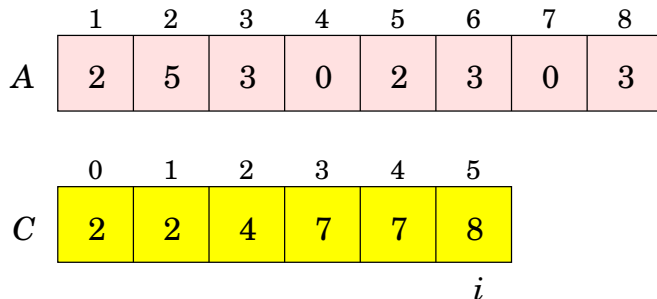
Exemplo de execução do COUNTINGSORT

	1	2	3	4	5	6	7	8
<i>A</i>	2	5	3	0	2	3	0	3

	0	1	2	3	4	5
<i>C</i>	2	2	4	7	7	1

i

Exemplo de execução do COUNTINGSORT



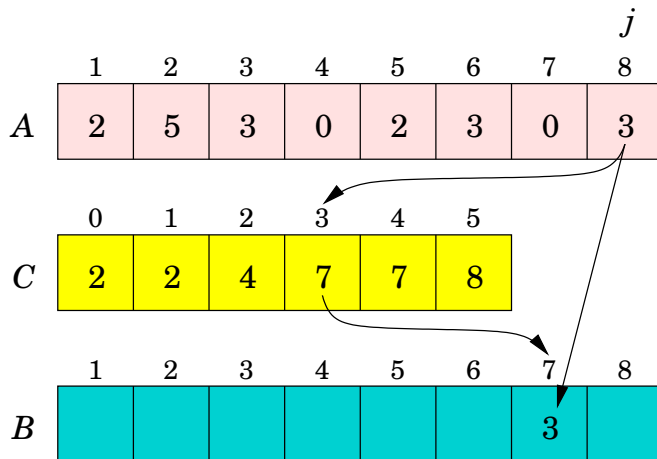
Exemplo de execução do COUNTINGSORT

	1	2	3	4	5	6	7	8
<i>A</i>	2	5	3	0	2	3	0	3

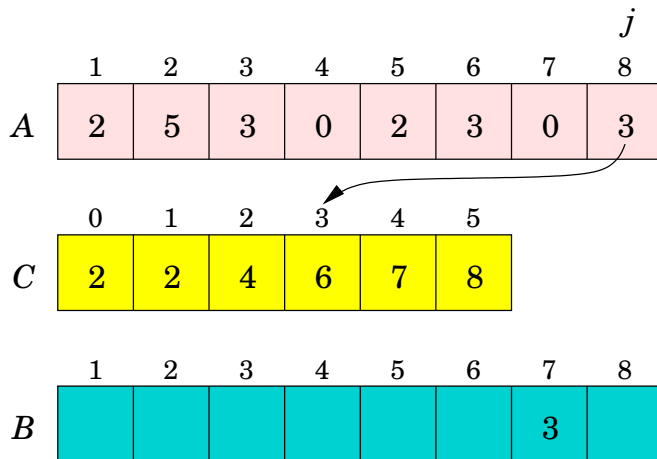
	0	1	2	3	4	5
<i>C</i>	2	2	4	7	7	8

i

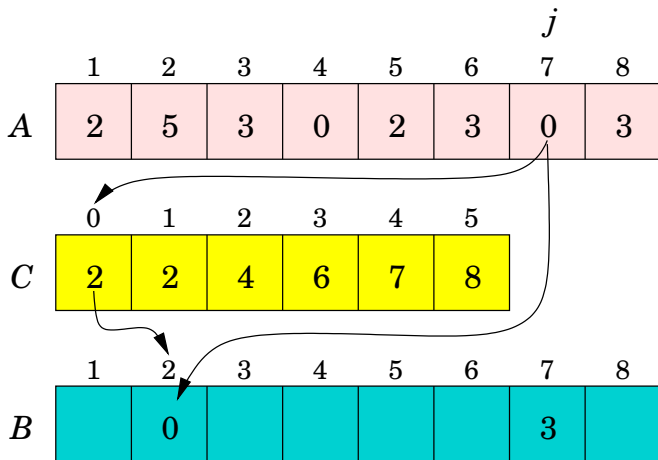
Exemplo de execução do COUNTINGSORT



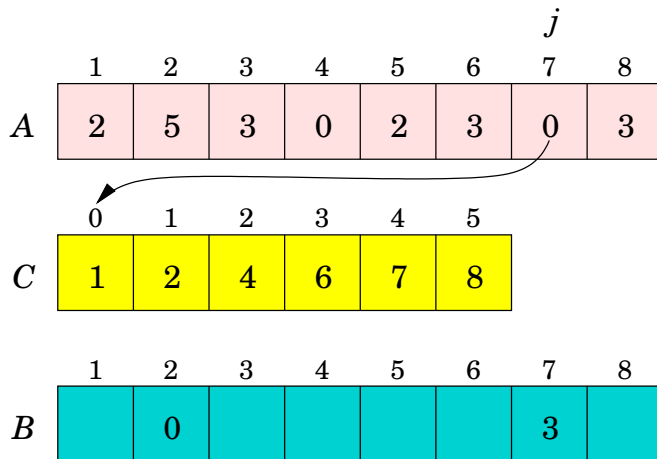
Exemplo de execução do COUNTINGSORT



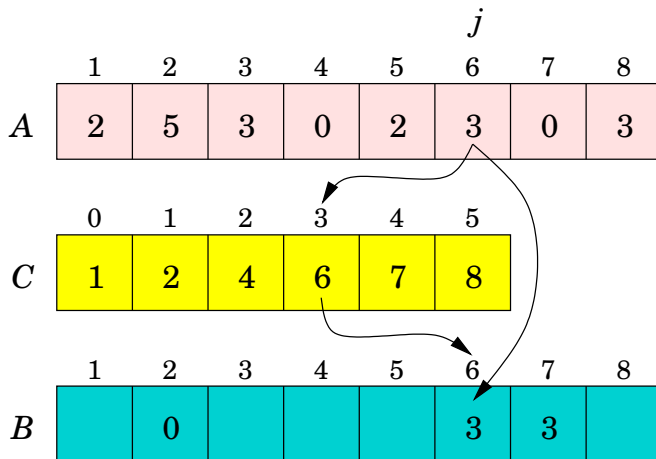
Exemplo de execução do COUNTINGSORT



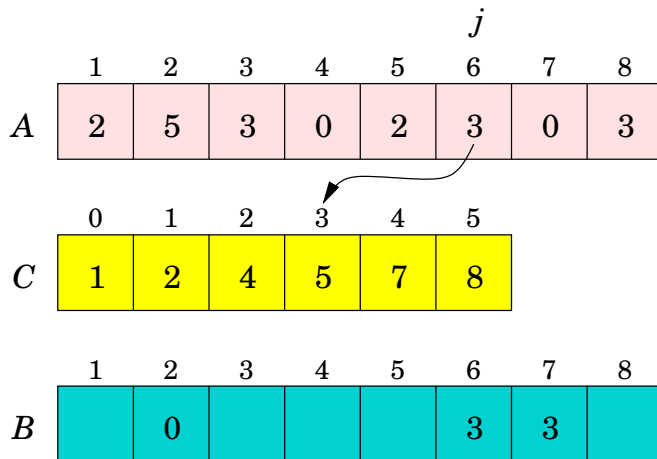
Exemplo de execução do COUNTINGSORT



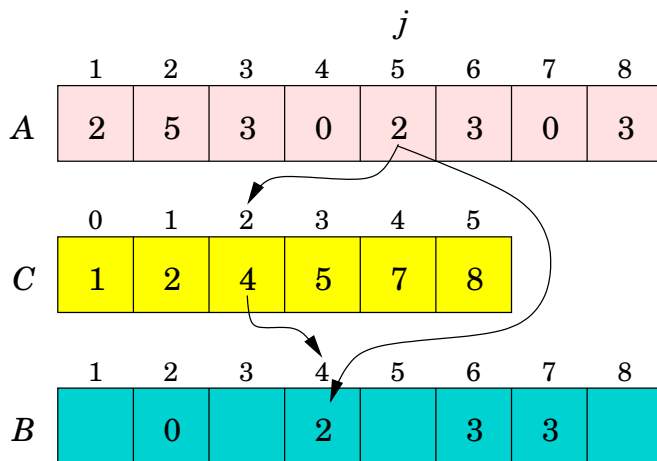
Exemplo de execução do COUNTINGSORT



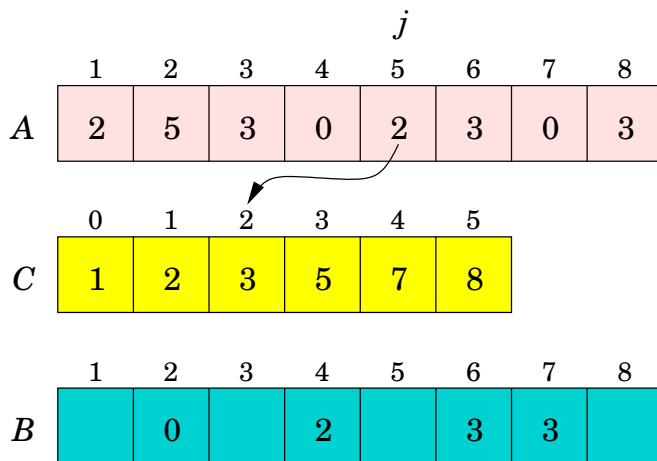
Exemplo de execução do COUNTINGSORT



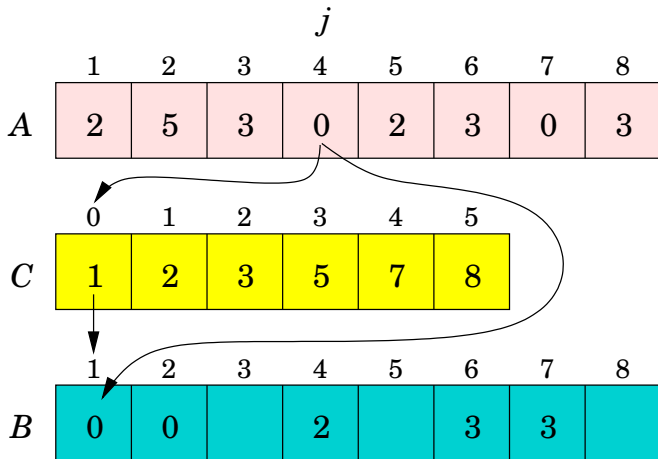
Exemplo de execução do COUNTINGSORT



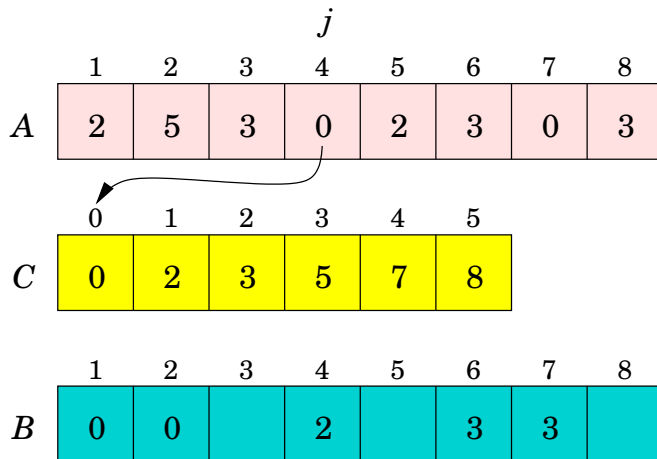
Exemplo de execução do COUNTINGSORT



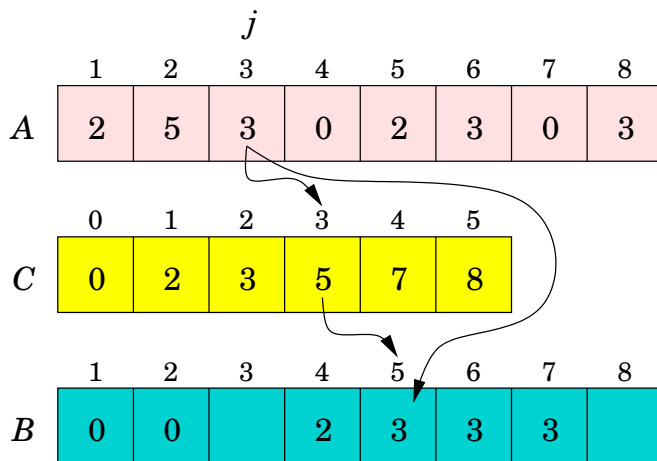
Exemplo de execução do COUNTINGSORT



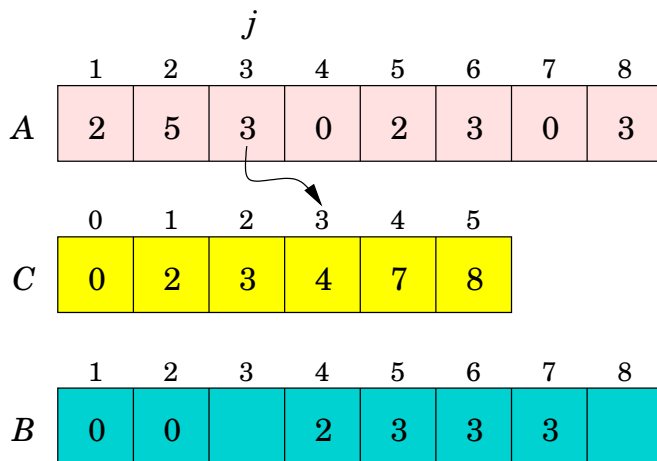
Exemplo de execução do COUNTINGSORT



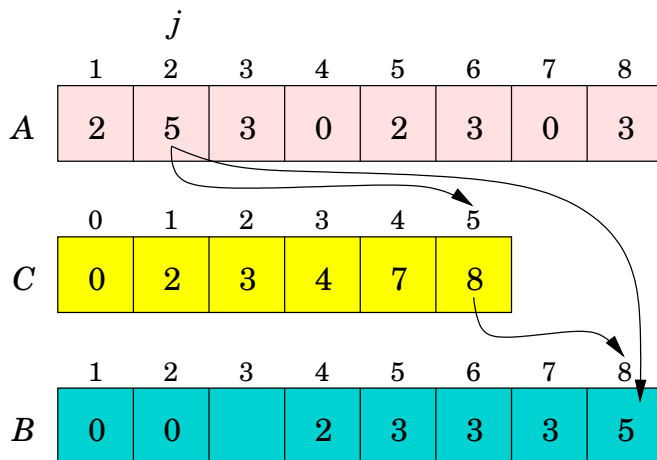
Exemplo de execução do COUNTINGSORT



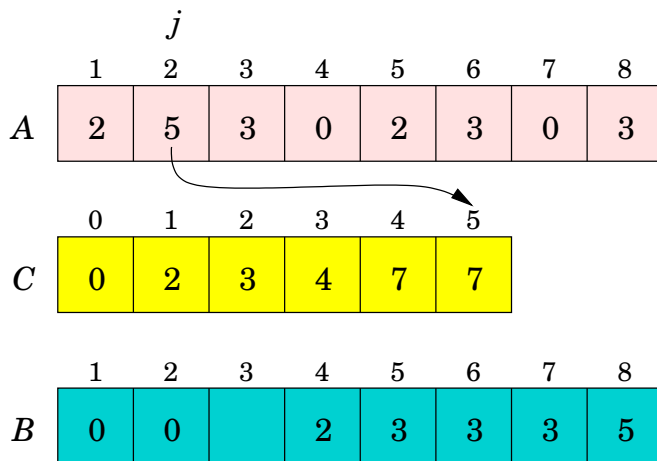
Exemplo de execução do COUNTINGSORT



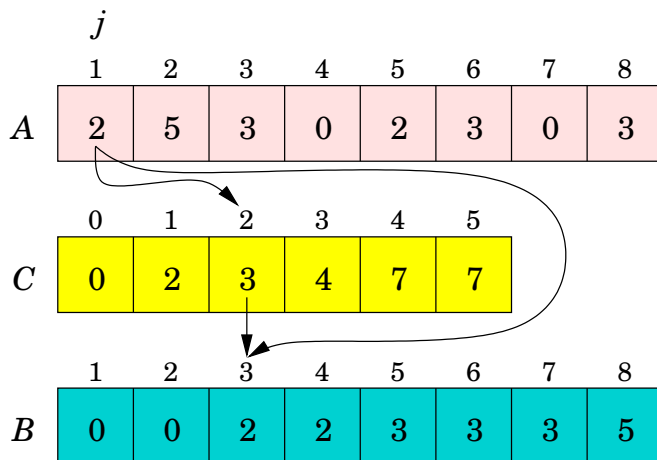
Exemplo de execução do COUNTINGSORT



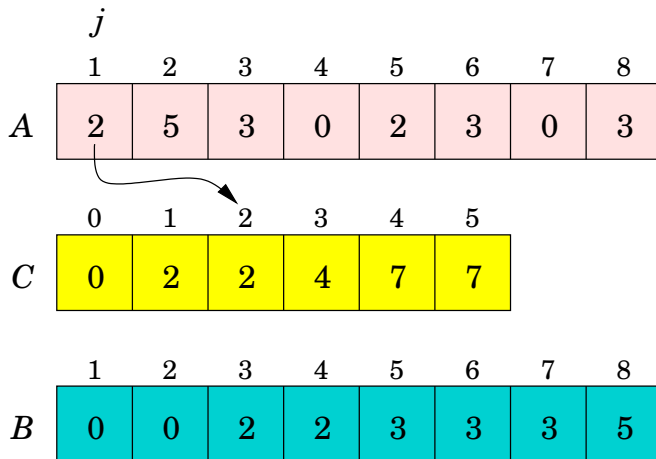
Exemplo de execução do COUNTINGSORT



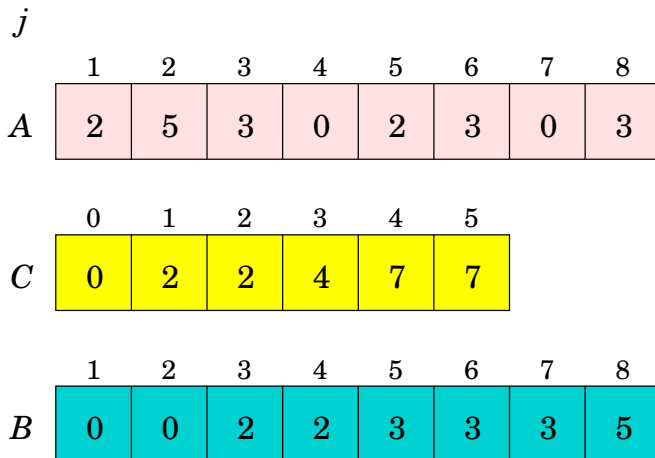
Exemplo de execução do COUNTINGSORT



Exemplo de execução do COUNTINGSORT



Exemplo de execução do COUNTINGSORT



Algoritmo COUNTINGSORT

COUNTINGSORT(A, B, k)

01. seja $C[0..k]$ um novo vetor

02. **para** $i \leftarrow 0$ **até** k

03. $C[i] \leftarrow 0$

04. **para** $j \leftarrow 1$ **até** $A.length$

05. $C[A[j]] \leftarrow C[A[j]] + 1$

06. // agora, $C[i]$ contém o número de elementos iguais a i

07. **para** $i \leftarrow 1$ **até** k

08. $C[i] \leftarrow C[i] + C[i - 1]$

09. // agora, $C[i]$ contém o número de elementos menores ou iguais a i

10. **para** $j \leftarrow A.length$ **decrecendo até** 1

11. $B[C[A[j]]] \leftarrow A[j]$

12. $C[A[j]] \leftarrow C[A[j]] - 1$

Tempo de execução do algoritmo COUNTINGSORT

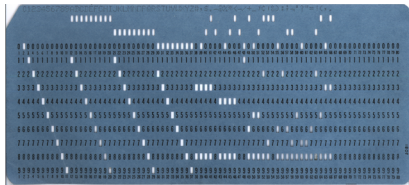
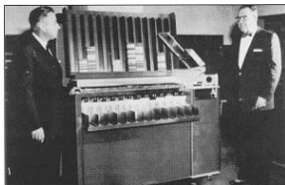
- ▶ estrutura de repetição das linhas 2–3 tem tempo de execução $\Theta(k)$
- ▶ estrutura de repetição das linhas 4–5 tem tempo de execução $\Theta(n)$
- ▶ estrutura de repetição das linhas 7–8 tem tempo de execução $\Theta(k)$
- ▶ estrutura de repetição das linhas 10–12 tem tempo de execução $\Theta(n)$
- ▶ assim, o tempo de execução do algoritmo COUNTINGSORT é $\Theta(k + n)$
- ▶ na prática, usamos o COUNTINGSORT quando $k = O(n)$ e assim, seu tempo de execução é $\Theta(n)$

Tempo de execução do algoritmo COUNTINGSORT

- ▶ COUNTINGSORT bate o limitante inferior $\Omega(n \lg n)$ porque não é um algoritmo de ordenação por comparação
- ▶ COUNTINGSORT é **estável**: elementos com mesmo valor aparecem no vetor de saída na mesma ordem em que aparecem no vetor de entrada
- ▶ RADIXSORT e BUCKETSORT são outros algoritmos de ordenação que têm tempo de execução linear

Preliminares

- ▶ algoritmo RADIXSORT era usado em máquinas que ordenavam cartões perfurados



Preliminares

- ▶ para dígitos decimais, usamos 10 dígitos em cada coluna
- ▶ um número com d dígitos usa d colunas
- ▶ o organizador de cartões pode trabalhar com uma coluna por vez
- ▶ cada uma das d colunas precisa ser ordenada
- ▶ ao invés de ordenar os dígitos de uma coluna fixa a partir do dígito mais significativo dos números de entrada, o `RADIXSORT` ordena primeiro a coluna contendo o dígito menos significativo, depois a coluna contendo o segundo dígito menos significativo, e assim sucessivamente, até a última coluna, que contém o dígito mais significativo do número
- ▶ é fundamental que o procedimento que ordena os dígitos de uma coluna seja **estável**

Preliminares

3 2 9

4 5 7

6 5 7

8 3 9

4 3 6

7 2 0

3 5 5

Preliminares

3 2 **9**

4 5 **7**

6 5 **7**

8 3 **9**

4 3 **6**

7 2 **0**

3 5 **5**

Preliminares

72**0**

35**5**

43**6**

45**7**

65**7**

32**9**

83**9**

Preliminares

7**2**0
3**5**5
4**3**6
4**5**7
6**5**7
3**2**9
8**3**9

Preliminares

7**2**0

3**2**9

4**3**6

8**3**9

3**5**5

4**5**7

6**5**7

Preliminares

720

329

436

839

355

457

657

Preliminares

329

355

436

457

657

720

839

Preliminares

3 2 9

3 5 5

4 3 6

4 5 7

6 5 7

7 2 0

8 3 9

Algoritmo RADIXSORT

$\text{RADIXSORT}(A, d)$

01. **para** $i \leftarrow 1$ **até** d
02. use um algoritmo de ordenação estável para ordenar o vetor A pelo dígito i

Tempo de execução do algoritmo RADIXSORT

Lema

Dados n números de d dígitos, nos quais cada dígito pode assumir até k valores possíveis, RADIXSORT ordena corretamente esses números em tempo $\Theta(d(n+k))$ se o algoritmo de ordenação estável usado tem tempo $\Theta(n+k)$.

- ▶ usa COUNTINGSORT para ordenar cada coluna
- ▶ quando d é constante e $k = O(n)$, o tempo de execução do RADIXSORT é $\Theta(n)$

Considerações

- ▶ `RADIXSORT` é usado mais frequentemente para ordenar registros de dados com mais de um campo chave (por exemplo, ano, mês, dia)
- ▶ se d é constante (por exemplo, 8 (bits)) e $k = O(n)$ então o tempo de execução do `RADIXSORT` é $\Theta(n)$, mas o uso do `COUNTINGSORT` faz com que o algoritmo necessite de espaço de memória auxiliar (não constante)

Introdução

- ▶ entrada é tomada a partir de uma distribuição uniforme
- ▶ tempo de execução no caso médio $O(n)$
- ▶ como o COUNTINGSORT, assume algo sobre a entrada: a entrada é gerada por um processo aleatório que distribui os elementos uniforme e independentemente sobre o intervalo $[0,1)$

Introdução

- ▶ divide o intervalo $[0,1)$ em n subintervalos de mesmo tamanho, chamados ***buckets***, e distribui os n números de entrada nesses *buckets*
- ▶ como os elementos de entrada são distribuídos uniforme e independentemente sobre $[0,1)$, esperamos que poucos números caiam em cada *bucket*
- ▶ a saída é produzida pela ordenação dos elementos em cada *bucket* e no percurso em ordem pelos *buckets*, listando seus elementos

Introdução

- ▶ o algoritmo assume que a entrada é um vetor A com n elementos e que cada elemento $A[i]$ satisfaz $0 \leq A[i] < 1$
- ▶ o código usa um vetor auxiliar $B[0..n-1]$ de listas ligadas (*buckets*) e assume que existem procedimentos usuais para manter tais listas

Algoritmo BUCKETSORT

BUCKETSORT(A)

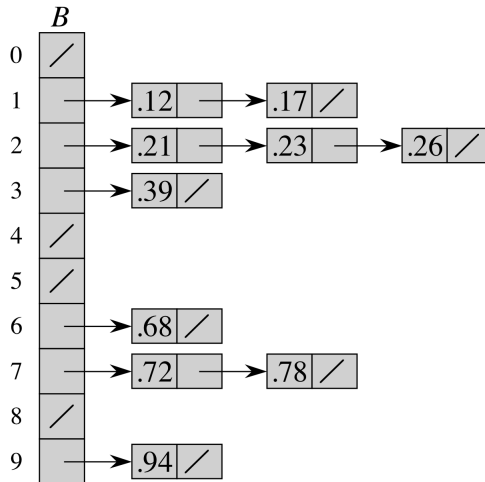
01. $n \leftarrow A.length$
02. seja $B[0..n-1]$ um novo vetor
03. **para** $i \leftarrow 0$ **até** $n-1$
04. inicialize uma lista vazia em $B[i]$
05. **para** $i \leftarrow 1$ **até** n
06. insira $A[i]$ na lista $B[\lfloor n A[i] \rfloor]$
07. **para** $i \leftarrow 0$ **até** $n-1$
08. ordene a lista $B[i]$ com o algoritmo INSERTIONSORT
09. concatene as listas $B[0], B[1], \dots, B[n-1]$ em ordem

Algoritmo BUCKETSORT

A

1	.78
2	.17
3	.39
4	.26
5	.72
6	.94
7	.21
8	.12
9	.23
10	.68

(a)



(b)

Ideia da correção

- ▶ considere dois elementos $A[i]$ e $A[j]$
- ▶ suponha sem perda de generalidade que $A[i] \leq A[j]$
- ▶ como $\lfloor n A[i] \rfloor \leq \lfloor n A[j] \rfloor$, então ou $A[i]$ é adicionado ao mesmo *bucket* de $A[j]$, ou é adicionado a um *bucket* de índice menor
- ▶ se $A[i]$ e $A[j]$ vão para o mesmo *bucket*, então a estrutura de repetição **para** das linhas 7–8 ordena tais elementos adequadamente
- ▶ se $A[i]$ e $A[j]$ vão para *buckets* diferentes, então a linha 9 ordena esses elementos corretamente

Tempo de execução

- ▶ todas as linhas do algoritmo, exceto a linha 8, têm tempo de execução de pior caso $O(n)$
- ▶ é necessário analisar o tempo total das n chamadas ao INSERTIONSORT na linha 8
- ▶ para analisar o custo das chamadas ao INSERTIONSORT, seja n_i a variável aleatória que denota o número de elementos adicionados ao *bucket* $B[i]$
- ▶ como o INSERTIONSORT tem tempo de execução quadrático, o tempo de execução do BUCKETSORT é

$$T(n) = \Theta(n) + \sum_{i=0}^{n-1} O(n_i^2).$$

Tempo de execução

- ▶ para analisar o caso médio do tempo de execução do `BUCKETSORT`, computamos o valor esperado do tempo de execução, onde a esperança é tomada sobre a distribuição de entrada
- ▶ tomando a esperança de ambos os lados e usando a linearidade da esperança, temos

$$\begin{aligned} E[T(n)] &= E\left[\Theta(n) + \sum_{i=0}^{n-1} O(n_i^2)\right] \\ &= \Theta(n) + \sum_{i=0}^{n-1} E\left[O(n_i^2)\right] \\ &= \Theta(n) + \sum_{i=0}^{n-1} O\left(E\left[n_i^2\right]\right). \end{aligned} \tag{1}$$

Tempo de execução

Lema

$$E[n_i^2] = 2 - 1/n, \text{ para } i = 0, 1, \dots, n-1.$$

Prova. Definimos inicialmente variáveis aleatórias indicadoras

$$X_{ij} = I\{A[j] \text{ é adicionado ao bucket } i\}$$

para todo $i = 0, 1, \dots, n-1$ e $j = 1, 2, \dots, n$. Assim,

$$n_i = \sum_{j=1}^n X_{ij}.$$

Tempo de execução

Prova (continuação).

Para computar $E[n_i^2]$, expandimos o quadrado e reagrupamos termos:

$$\begin{aligned}
 E[n_i^2] &= E\left[\left(\sum_{j=1}^n X_{ij}\right)^2\right] \\
 &= E\left[\sum_{j=1}^n \sum_{k=1}^n X_{ij} X_{ik}\right] \\
 &= E\left[\sum_{j=1}^n X_{ij}^2 + \sum_{1 \leq j \leq n} \sum_{\substack{1 \leq k \leq n \\ k \neq j}} X_{ij} X_{ik}\right] \\
 &= \sum_{j=1}^n E[X_{ij}^2] + \sum_{1 \leq j \leq n} \sum_{\substack{1 \leq k \leq n \\ k \neq j}} E[X_{ij} X_{ik}], \tag{2}
 \end{aligned}$$

Tempo de execução

Prova (continuação).

A última igualdade segue da linearidade da esperança. Avaliamos os dois somatórios separadamente. A variável aleatória indicadora X_{ij} vale 1 com probabilidade $1/n$ e 0 caso contrário e assim

$$\begin{aligned} E[X_{ij}^2] &= 1^2 \cdot \frac{1}{n} + 0^2 \cdot \left(1 - \frac{1}{n}\right) \\ &= \frac{1}{n}. \end{aligned}$$

Tempo de execução

Prova (continuação).

Quando $k \neq j$, as variáveis X_{ij} e X_{ik} são independentes, e portanto

$$\begin{aligned} E[X_{ij} X_{ik}] &= E[X_{ij}] E[X_{ik}] \\ &= \frac{1}{n} \cdot \frac{1}{n} \\ &= \frac{1}{n^2}. \end{aligned}$$

Tempo de execução

Prova (continuação).

Substituindo esses valores esperados na equação (2) temos

$$\begin{aligned}
 E[n_i^2] &= \sum_{j=1}^n E[X_{ij}^2] + \sum_{1 \leq j \leq n} \sum_{\substack{1 \leq k \leq n \\ k \neq j}} E[X_{ij} X_{ik}] \\
 &= \sum_{j=1}^n \frac{1}{n} + \sum_{1 \leq j \leq n} \sum_{\substack{1 \leq k \leq n \\ k \neq j}} \frac{1}{n^2} \\
 &= n \cdot \frac{1}{n} + n(n-1) \cdot \frac{1}{n^2} \\
 &= 1 + \frac{n-1}{n} \\
 &= 2 - \frac{1}{n}.
 \end{aligned}$$

□

Tempo de execução

- ▶ usando esse valor esperado na equação (1), concluímos que o tempo de execução no caso médio para o algoritmo `BUCKETSORT` é

$$\begin{aligned} E[T(n)] &= \Theta(n) + \sum_{i=0}^{n-1} O(E[n_i^2]) \\ &= \Theta(n) + n \cdot O\left(2 - \frac{1}{n}\right) \\ &= \Theta(n). \end{aligned}$$

Exercícios

- 8.1-1 Qual é a menor profundidade possível de uma folha em uma árvore de decisão para um algoritmo de ordenação por comparação?
- 8.1-2 Use a aproximação de Stirling para obter limitantes justos para $\lg(n!)$. A aproximação de Stirling é

$$n! = \sqrt{2\pi n} \left(\frac{n}{e}\right)^n \left(1 + \Theta\left(\frac{1}{n}\right)\right).$$

Exercícios

- 8.2-1 Ilustre a operação do COUNTINGSORT sobre o vetor $A = \langle 6, 0, 2, 0, 1, 3, 4, 6, 1, 3, 2 \rangle$.
- 8.2-2 Prove que o COUNTINGSORT é estável.
- 8.2-3 Suponha que reescrevemos a sentença **para** da linha 10 do COUNTINGSORT da seguinte forma:

10. **para** $j \leftarrow 1$ **até** $A.length$

Mostre que o algoritmo ainda assim realiza a ordenação dos elementos corretamente. O algoritmo modificado é estável?

- 8.2-4 Descreva um algoritmo que, dados n números inteiros no intervalo de 0 a k , pré-processa a entrada e então responde qualquer consulta sobre quantos dos n inteiros estão no intervalo $[a, b]$ em tempo $O(1)$. Seu algoritmo deve usar tempo $\Theta(n + k)$ para o pré-processamento.

Exercícios

- 8.3-1 Ilustre a operação do `RADIXSORT` sobre a seguinte lista de palavras da língua portuguesa: AMA, JAZ, FEZ, SOB, ERA, TEM, MAU, ATA, VIA, UVA, ATO, DOR, MAL, EGO, FOR, TER, PAZ, FOZ, DOM, MAS, VIL, OPA, POR, RIO, VIR.
- 8.3-2 Quais dos seguintes algoritmos de ordenação são estáveis: `INSERTIONSORT`, `MERGESORT`, `HEAPSORT` e `QUICKSORT`. Forneça um esquema simples que torne estável qualquer algoritmo de ordenação. Quanto de tempo e espaço adicional seu esquema requer?
- 8.3-3 Use indução para provar que o `RADIXSORT` está correto. Onde sua prova necessita da suposição que o algoritmo de ordenação intermediário é estável?
- 8.3-4 Mostre como ordenar n números inteiros no intervalo de 0 até $n^2 - 1$ em tempo $O(n)$.

Exercícios

- 8.4-1 Ilustre a operação do `BUCKETSORT` sobre o vetor $A = \langle .79, .13, .16, .64, .39, .20, .89, .53, .71, .42 \rangle$.
- 8.4-2 Explique por que o tempo de execução de pior caso do `BUCKETSORT` é $\Theta(n^2)$. Que mudança simples no algoritmo preserva seu tempo de execução de caso médio e faz seu tempo de execução de pior caso $O(n \lg n)$?
- 8.4-3 Seja X uma variável aleatória cujo valor é igual ao número de caras resultantes quando lançamos duas moedas honestas. O que é $E[X^2]$? O que é $E^2[X]$?